CS-114 Fundamentals of Programming

Arrays

Instructor: Jahan Zeb
Department of Computer & Software Engineering (DC&SE)
College of E&ME
NUST

Introduction

- Arrays
  - Structures of related data items
  - Static entity (same size throughout program)

Arrays

- Array
  - Consecutive group of memory locations
  - Same name and type (**`int`**, **`char`**, etc.)
- To refer to an element
  - Specify array name and position number (index)
  - Format: arrayname[position number]
  - First element at position 0
- N-element array c

  `c[ 0 ]`, **`c[ 1 ]`** … **`c[ n - 1 ]`**
  - Nth element as position N-1

Arrays

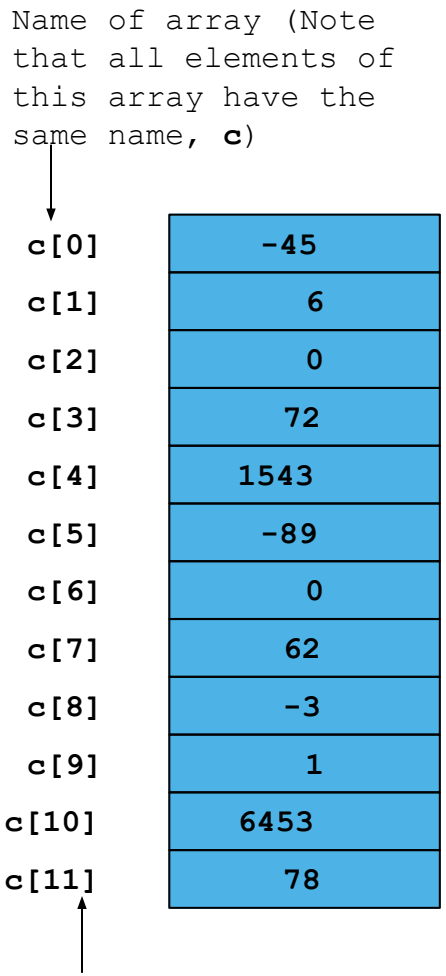
- Array elements like other variables
  - Assignment, printing for an integer array **c**

```
c[ 0 ] =  3;
cout << c[ 0 ];
```

- Can perform operations inside subscript

  c[5 - 2] same as **c[3]**

Arrays


Name of array (Note that all elements of this array have the same name, c)
c[0]
-45
c[1]
6
c[2]
0
c[3]
72
c[4]
1543
c[5]
-89
c[6]
0
c[7]
62
c[8]
-3
c[9]
1
c[10]
6453
c[11]
78
Position number of the element within array c

Declaring Arrays

- When declaring arrays, specify
  - Name
  - Type of array
    - Any data type
  - Number of elements
  - *type arrayName* **[** *arraySize* **];**

```
int c[ 10 ];  // array of 10 integers
float d[ 3284 ]; // array of 3284 floats
```

- Declaring multiple arrays of same type
  - Use comma separated list, like regular variables

```
int b[ 100 ], x[ 27 ];
```

Examples Using Arrays

- Initializing arrays
  - For loop
    - Set each element
  - Initializer list
    - Specify each element when array declared

```
int n[ 5 ] = { 1, 2, 3, 4, 5 };
```

    - If not enough initializers, rightmost elements 0
  - To set every element to same value

```
int n[ 5 ] = { 0 };
```

  - If array size omitted, initializers determine size

```
int n[] = { 1, 2, 3, 4, 5 };
```

    - 5 initializers, therefore 5 element array

```
1
2  // Initializing an array.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <iomanip>
9
10 using std::setw;
11
12 int main()
13 {
14    int n[ 10 ]; // n is an array of
15
16    // initialize elements of array n
17    for ( int i = 0; i < 10; i++ )
18       n[ i ] = 0;   // set element at location i to 0
19
20    cout << "Element" << setw( 13 ) << "Value" << endl;
21
22    // output contents of array n in tabular format
23    for ( int j = 0; j < 10; j++ )
24       cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;
25
```

Declare a 10-element array of integers.

Initialize array to **0** using a for loop. Note that the array has elements **n[0]** to **n[9]**.

```
26    return 0;  // indicates successful termination
27
28 } // end main
```

```
Element         Value
      0             0
      1             0
      2             0
      3             0
      4             0
      5             0
      6             0
      7             0
      8             0
      9             0
```

```
1
2  // Initializing an array with a declaration.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <iomanip>
9
10 using std::setw;
11
12 int main()
13 {
14    // use initializer list to initialize array n
15    int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };
16
17    cout << "Element" << setw( 13 ) << "Value" << endl;
18
19    // output contents of array n in tabular format
20    for ( int i = 0; i < 10; i++ )
21       cout << setw( 7 ) << i << setw( 13 ) << n[ i ] << endl;
22
23    return 0;  // indicates successful termination
24
25 } // end main
```

Note the use of the initializer list.

Element	Value
0	32
1	27
2	64
3	18
4	95
5	14
6	90
7	70
8	60
9	37

Examples Using Arrays

- Array size
  - Can be specified with constant variable (**const**)
    - **const int size = 20;**
  - Constants cannot be changed
  - Constants must be initialized when declared
  - Also called named constants or read-only variables

```
1
2  // Initialize array s to the even integers from 2 to 20.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <iomanip>
9
10 using std::setw;
11
12 int main()
13 {
14    // constant variable can be used to
15    const int arraySize = 10;
16
17    int s[ arraySize ];  // array s has 10 el
18
19    for ( int i = 0; i < arraySize; i++ )  //
20       s[ i ] = 2 + 2 * i;
21
22    cout << "Element" << setw( 13 ) << "Value
23
```

Note use of **`const`** keyword. Only **`const`** variables can specify array sizes.

The program becomes more scalable when we set the array size using a **`const`** variable. We can change **`arraySize`**, and all the loops will still work (otherwise, we'd have to update every loop in the program).

```
24    // output contents of array s in tabular format
25    for ( int j = 0; j < arraySize; j++ )
26       cout << setw( 7 ) << j << setw( 13 ) << s[ j ] << endl;
27
28    return 0;  // indicates successful termination
29
30 } // end main
```

```
Element         Value
      0             2
      1             4
      2             6
      3             8
      4            10
      5            12
      6            14
      7            16
      8            18
      9            20
```

```
1
2  // Using a properly initialized constant variable.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  int main()
9  {
10    const int x = 7;  // initialized constant variable
11
12    cout << "The value of constant variable x is: "
13         << x << endl;
14
15    return 0;  // indicates successful termination
16
17 } // end main
```

Proper initialization of **`const`** variable.

```
The value of constant variable x is: 7
```

```
1
2  // A const object must be initi
3
4  int main()
5  {
6     const int x;  // Error: x mu
7
8     x = 7;        // Error: cannot modify a const variable
9
10    return 0;     // indicates successful termination
11
12 } // end main
```

Uninitialized **`const`** results in a syntax error. Attempting to modify the **`const`** is another error.

```
d:\cpphtp4_examples\ch04\Fig04_07.cpp(6) : error C2734: 'x' :
   const object must be initialized
```

```
1
2  // Compute the sum of the elements of the array.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  int main()
9  {
10    const int arraySize = 10;
11
12    int a[ arraySize ] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
13
14    int total = 0;
15
16    // sum contents of array a
17    for ( int i = 0; i < arraySize; i++ )
18       total += a[ i ];
19
20    cout << "Total of array element values is " << total << endl;
21
22    return 0;  // indicates successful termination
23
24 } // end main
```

```
Total of array element values is 55
```

```
1
2  // Histogram printing program.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <iomanip>
9
10 using std::setw;
11
12 int main()
13 {
14    const int arraySize = 10;
15    int n[ arraySize ] = { 19, 3, 15, 7, 11, 9, 13, 5, 17, 1 };
16
17    cout << "Element" << setw( 13 ) << "Value"
18         << setw( 17 ) << "Histogram" << endl;
19
20    // for each element of array n, output a bar in histogram
21    for ( int i = 0; i < arraySize; i++ ) {
22       cout << setw( 7 ) << i << setw( 13 )
23            << n[ i ] << setw( 9 );
24
25       for ( int j = 0; j < n[ i ]; j++ )   // print one bar
26          cout << '*';
```

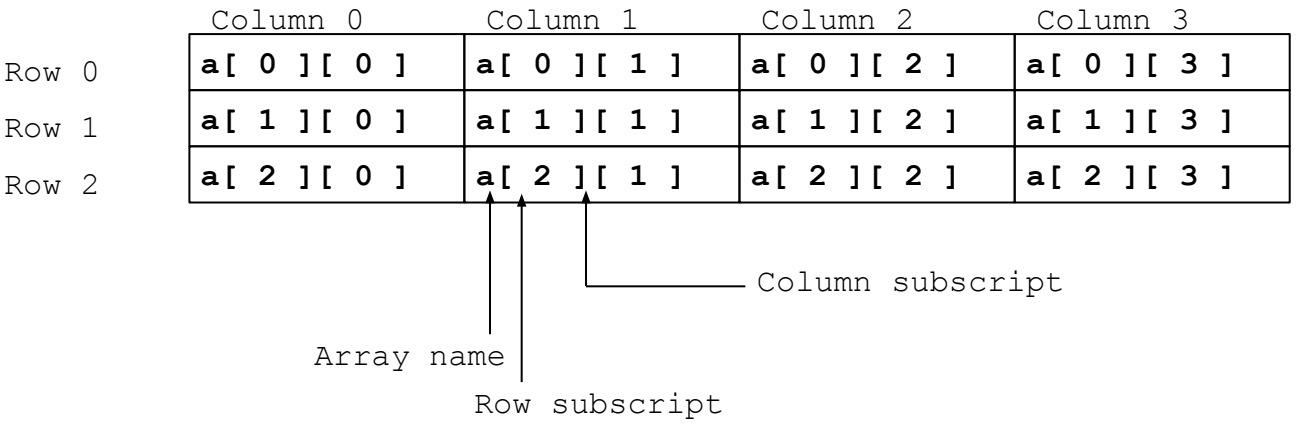
Prints asterisks corresponding to size of array element, **`n[i]`**.

```
27
28        cout << endl;  // start next line of output
29
30     } // end outer for structure
31
32     return 0;  // indicates successful termination
33
34  } // end main
```

```
Element         Value         Histogram
      0            19         ********************
      1             3         ***
      2            15         ***************
      3             7         *******
      4            11         ***********
      5             9         *********
      6            13         *************
      7             5         *****
      8            17         *****************
      9             1         *
```

Multiple-Subscripted Arrays

- Multiple subscripts
  - **a[i][j]**
  - Tables with rows and columns
  - Specify row, then column
  - "Array of arrays"
    - **a[0]** is an array of 4 elements
    - **a[0][0]** is the first element of that array



Multiple-Subscripted Arrays

- To initialize
  - Default of **0**
  - Initializers grouped by row in braces

```
int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };
```

Row 0 **Row 1**

1	2
3	4

```
int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };
```

1	0
3	4

Multiple-Subscripted Arrays

- Referenced like normal

```
cout << b[ 0 ][ 1 ];
```

1	0
3	4

  - Outputs **0**
  - Cannot reference using commas

```
cout << b[ 0, 1 ];
```

    - Syntax error

- Function prototypes
  - Must specify sizes of subscripts
    - First subscript not necessary, as with single-scripted arrays
  - **`void printArray( int [][ 3 ] );`**

```
1
2  // Initializing multidimensional arrays.
3  #include <iostream>
4
5
6
7
8
9
10 int main()
11 {
12    int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
13    int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
14    int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };
15
16    cout << "Values in array1 by row are:" << endl;
```

Note the format of the prototype.

Note the various initialization styles. The elements in **`array2`** are assigned to the first row and then the second.

```
28
32    for ( int i = 0; i < 2; i++ ) {     // fo
33
34       for ( int j = 0; j < 3; j++ )    // ou
35          cout << array1[ i ][ j ] << ' ';
36
37       cout << endl;  // start new line of output
38
39    } // end outer for structure
40
41
```

For loops are often used to iterate through arrays. Nested loops are helpful with multiple-subscripted arrays.

```
Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0
```